

北商周邊 YouBike 高需求時段預測

Python爬蟲與資料分析分組專案 第02組

組長：賴子渝 組員：王秀方 指導老師：江尚瑀 老師



我們想回答什麼？

結合北商周邊站點的歷史借車紀錄與同期天氣資料，能否判斷某站點在特定日期與時段是否屬於相對高需求？

- 我們預測的是某站點某時段的「相對需求高低」，帮大家提早掌握哪些時段借車人比較多。
- 研究範圍是北商臺北校區中心方圓500公尺直線距離內，總共21個站點。
- 資料期間是2025年12月1日到2026年2月28日，總共三個月。

別人也是這樣做的嗎？

2016

紐約的研究

2016年紐約Citi Bike的研究指出，站點需求存在空間與時間上的差異，氣象與時間屬性可以納入需求模型。

2017

多倫多的研究

2017年多倫多的研究發現，站點層級資料顯示天氣與土地使用等因素，跟共享單車使用量有關。

2019

舊金山灣區的研究

2019年舊金山灣區的研究用Random Forest排序特徵，結果顯示時段、溫度與濕度是重要的預測因子。

- 這三篇研究共同支持的是：站點、時間與天氣可以作為共享單車需求分析的合理特徵，本專題依照這個方向設計特徵與模型。
- 我們參考的是這三篇研究建立特徵的邏輯，本專題的模型與結論仍以北商周邊三個月的實際資料為準。

我們怎麼取得資料？

```
step01_youbike_feasibility_check.py

1  # 以串流方式下載，避免大檔案一次載入記憶體；設定 timeout 避免政府主機回應緩慢時卡住
2  with requests.get(url, stream=True, timeout=180,
3                    headers={"User-Agent": "NTUB-YouBike-Course-Feasibility-Test/1.0"}) as response:
4      # 非 200 狀態碼時立即拋出例外，不悄悄略過失敗的下載
5      response.raise_for_status()
6      # 開檔為二進位寫入模式，把原始檔完整保留下來以利之後稽核
7      with destination.open("wb") as output:
8          # 每次讀取 1MB，逐塊寫入磁碟，而不是一次讀取整個回應內容
9          for block in response.iter_content(chunk_size=1024 * 1024):
10             if block:
11                 output.write(block)
12             # 回傳 HTTP 狀態碼，讓呼叫端可以記錄到稽核報告中
13     return response.status_code
```

1

我們用Python直接呼叫官方公開下載連結與Open-Meteo天氣API取得資料，全程自動化、可重複執行，也不需要登入或金鑰。

2

三個月月份不是我們挑的，是程式自動讀取官方公布的可下載月份清單，抓取最新的3個可用月份，確保流程可重跑、可驗證。

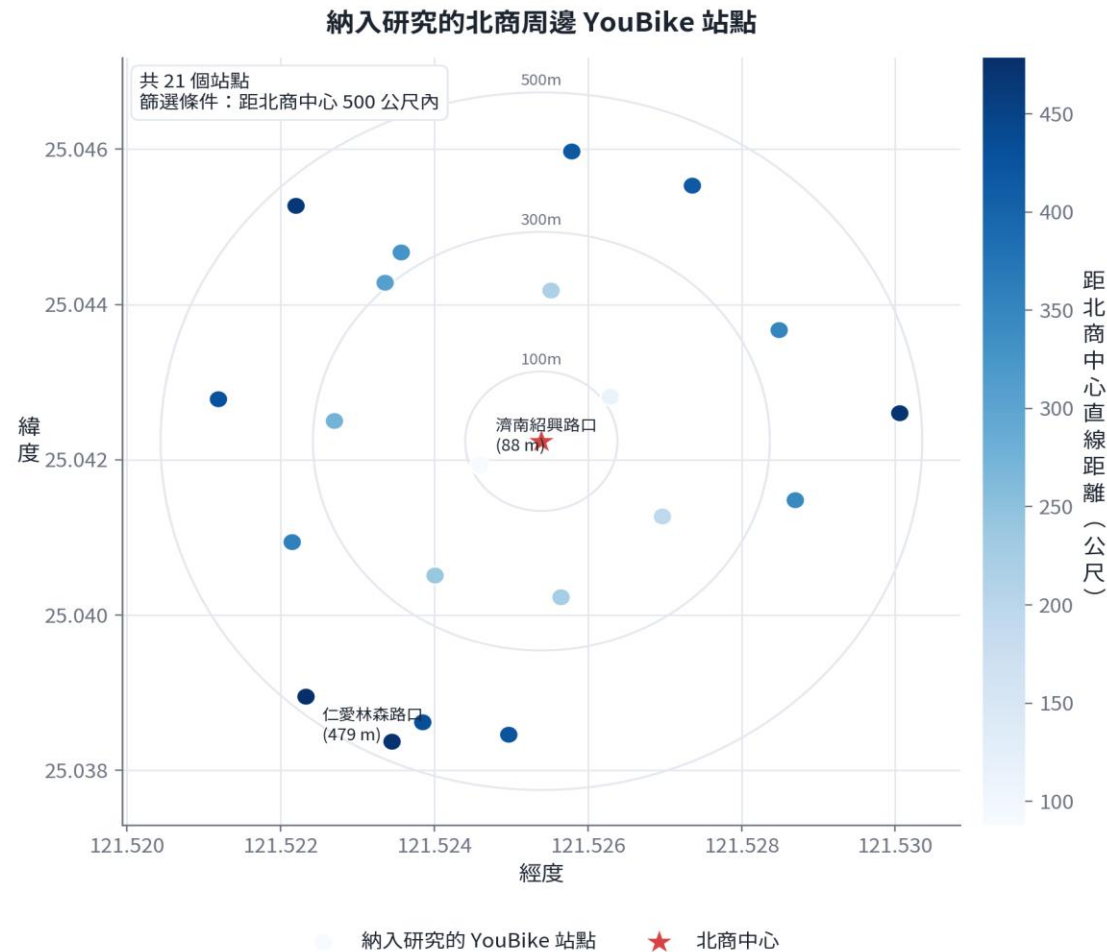
我們怎麼取得資料？(續)

3

第一個限制是站點範圍：我們用YouBike「目前」的即時站點位置，回推「過去三個月」哪些站算在北商周邊500公尺內。

4

第二個限制是天氣座標誤差：查詢座標是北商中心，但Open-Meteo實際回傳的是最近的氣象網格點，跟北商中心相差約2公里。這是氣象資料服務常見的做法，同一氣候區域內溫度降雨型態的差異通常不大。



附注。點的颜色深浅對應各站距北商中心的直線距離 (公尺)，顏色越深代表距離越遠；星號為北商中心位置，非站點；淡灰色同心圓為 100/300/500 公尺參考輻射圈，用以呈現所有站點皆落在 500 公尺篩選範圍內。

借車紀錄和天氣資料怎麼變成同一張表？

借車事件（單筆）→ 彙整成「站點×小時」的借車次數 → 用「站名 + 時間」當鍵值合併天氣資料 → 一行代表一個站點在某個小時的需求與天氣

1

原始的借車紀錄是一筆一筆的事件資料，一個人借一次車就是一筆，我們先把它彙整成「這一站在這一個小時總共被借了幾次」，資料就從事件層級變成站點 - 小時層級。

2

天氣資料本身是「每小時一筆」，我們用「站名」加上「時間」這兩個欄位當作共同的鍵值，把彙整後的借車次數和同一個小時的天氣資料合併成一行。

借車紀錄和天氣資料怎麼變成同一張表？(續)

資料處理各階段的規模變化：從單筆借車事件，一路彙整成站點一小時的模型資料



附注。數字為累計列數；前兩階段單位為單筆借車事件，第三階段起改以「站點一小時」為分析單位，因此列數大幅下降是統計單位改變所致，不是資料被捨棄或遺失。

3

這張圖顯示的是資料筆數在每個處理階段怎麼變化：

從最原始的借車事件，到彙整成站點 - 小時，筆數會大幅減少，這是彙整方式造成的，不是資料流失，因為最後一格已經不是同一種計數單位了。

4

合併之後，每一行資料就代表「某個站點、某一個小時」的借車次數，加上同一時間點的溫度、降雨量、風速等天氣狀況，這就是後面模型要用的資料表雛形。

合併是怎麼寫出來的？結果長什麼樣？

```
step02_demand_weather_merge.py

1  # 原始借車事件先依「站名比對鍵+時間（小時）」彙整加總，從單筆事件變成站點一小時的借車次數
2  event_counts = pd.concat(all_counts, ignore_index=True)
3  event_counts = event_counts.groupby(["站名比對鍵", "時間"], as_index=False)["借車次數"].sum()
4
5  # 建立完整的站點×小時骨架（就算某站某小時沒有借車事件，也要保留這一行，之後補0）
6  all_hours = pd.date_range(start_time, end_time, freq="h")
7  demand = pd.MultiIndex.from_product(
8      [nearby["站名比對鍵"], all_hours], names=["站名比對鍵", "時間"]
9  ).to_frame(index=False)
10
11 # 用 left join 把彙整後的借車次數併回完整骨架；沒配對到的列會是 NaN，代表這一站這一小時沒有借車事件
12 demand = demand.merge(event_counts, on=["站名比對鍵", "時間"], how="left")
13 demand["借車次數"] = demand["借車次數"].fillna(0).astype(int)
14
15 # 再用「時間」把每小時的天氣資料併回來；天氣是全站共用的同一份序列，不分站點
16 merged = demand.merge(weather, on="時間", how="left")
```

1 程式碼分成三步：第一步把借車事件依「站點加時間」分組加總，變成站點 - 小時的借車次數；第二步建立完整的站點乘小時空表，確保沒有借車事件的小時也保留、不會漏掉；第三步把彙整後的借車次數，跟依時間合併的天氣資料，都併到同一張表。

2 沒有配對到借車事件的小時，合併之後會为空值，我們統一補為0，因為原始資料是事件紀錄，沒出現就代表真實發生的借車次數是0次。

合併是怎麼寫出來的？結果長什麼樣？(續)

3

下方是合併完成後、實際存在的資料截圖：可以看到「站點」「時間」是合併用的兩個依據欄位，「借車次數」是借車事件彙整而來，右邊幾欄是溫度、降雨量、風速，這些是天氣資料合併進來的，同一行同時看得到借車量和天氣狀況。

●●● 北商YouBike_站點相對高需求模型資料_測試.csv

站名比對鍵 (合併鍵之一)	時間 (合併鍵之二)	借車次數 (借車事件彙整而來)	temperature_2m (天氣資料併入)	precipitation (天氣資料併入)	wind_speed_10m (天氣資料併入)
捷運善導寺站3號出口忠孝東路側	2025-12-01 07:00:00	23	18.7	1.2	1.1
捷運善導寺站3號出口忠孝東路側	2025-12-01 08:00:00	16	19.7	0.1	2.7
捷運善導寺站3號出口忠孝東路側	2025-12-01 09:00:00	14	21.1	0.0	0.6
捷運善導寺站3號出口忠孝東路側	2025-12-01 10:00:00	12	22.6	0.0	1.1

4

這張截圖是實際模型資料裡的4筆，站點是借車量最高的捷運善導寺站3號出口，時間是12月1日的通勤尖峰時段，可以看到借車次數確實從23次逐漸降到12次，跟溫度、降雨量的變化並列在同一行。

資料處理做了哪些事？

```
step01_youbike_feasibility_check.py

1  def normalize_station_name(value: object) -> str:
2      # 將站名轉為半形、小寫，統一比對基準（NFKC 會把全形字元轉成對應半形）
3      text = unicodedata.normalize("NFKC", str(value) if pd.notna(value) else "").lower()
4      # 「臺」與「台」是同一個字的兩種寫法，全部統一成「台」
5      text = text.replace("臺", "台")
6      # 移除「YouBike 2.0」這類系統版本前綴，不同資料來源標示方式不一致
7      text = re.sub(r"youbike\s*2\.0", "", text)
8      # 只保留數字、英文字母與中文字，其餘符號（空格、括號等）一律去除
9      return re.sub(r"[^0-9a-z\u4e00-\u9fff]+", "", text)
```

1

我們先處理了站名統一的問題，包括全形半形、臺字台字、系統前綴的差異，避免同一站被當成不同站。

2

我們也處理了欄位型態與欄位篩選：原始借車紀錄的時間欄位是文字格式，我們轉換成標準時間格式才能用來排序、分組和合併；原始租借紀錄裡有很多欄位，我們只保留分析用得到的借車時間和借車站兩個欄位，其餘欄位一開始就不讀取，讓後續處理更單純。

資料處理做了哪些事？(續)

3

完整的站點 - 小時格如果沒有借車事件，就記為 0，因為原始資料是事件紀錄，沒出現就代表真實發生的借車次數是 0 次。

4

重複值的部分，我們以站點加小時作為分析鍵去檢查，實際稽核下來重複值是 0 筆。

5

IQR 檢查標記了 2,239 筆高值，占全部資料的 4.94%。我們評估這些是真實的尖峰需求，所以予以保留、不刪除。

6

這裡的 IQR 檢查是用「全部期間」的資料計算，目的只是盤點資料品質。



step02_demand_weather_merge.py

```

1  # 用 left join 把「有借車事件」的小時併回完整站點×小時骨架；沒配對到的列會是 NaN
2  demand = demand.merge(event_counts, on=["站名比對鍵", "時間"], how="left")
3  # 把 merge 後的 NaN（代表該站該小時沒有任何借車事件）補為 0，並轉回整數計數
4  demand["借車次數"] = demand["借車次數"].fillna(0).astype(int)
5  # 依時間把每小時天氣併回需求資料；天氣是站點共用的同一份序列，不分站點
6  merged = demand.merge(weather, on="時間", how="left")
7
8  # step05_data_quality_report_charts.py
9  # IQR（四分位距）= Q3 - Q1，用來衡量借車次數中間 50% 資料的離散程度
10 iqr = q3 - q1
11 # 高值判定門檻採 Tukey 規則：Q3 + 1.5 倍 IQR，超過此門檻視為統計上的高值
12 upper_bound = q3 + 1.5 * iqr
13 # 標記出高於門檻的列；這裡只標記、不刪除，因為高值可能是真實的尖峰需求
14 iqr_flagged = data["借車次數"] > upper_bound

```

建立了哪些新特徵？

01

星期幾、是否週末：這兩個特徵是從借車時間直接拆解出來的。

02

小時的週期特徵，也就是sin和cos：這樣的處理讓23時和0時在數值上被視為相鄰，避免深夜時段被誤判為數值差距很大。

03

溫度、降雨量、天氣現象代碼、風速：這四項都來自天氣資料。

什麼叫做『高需求』？

規則：借車次數 $\geq$ 該站訓練期80分位數（向上取整），即標記為「高需求」

1

每一站都使用「自己」的門檻，也就是各站分開計算，這樣可以避免大站小站直接比較總量。

2

80分位數是我們這組經過討論後選定的研究設計，老實說這個數字本身沒有標準答案。

3

70、85、90分位數的敏感度比較，我們列為第28頁未來改進方向之一，作為下一階段可以延伸的方向。

```
step03_high_demand_labeling.py

1 train = data.loc[data["資料切分"].eq("訓練期")].copy()
2 # 只用訓練期資料計算每站的門檻，測試期完全不參與門檻計算
3 thresholds = train.groupby("站名比對鍵")["借車次數"].quantile(QUANTILE)
4 thresholds = thresholds.rename("訓練期80分位數門檻").reset_index()
5 # 門檻取整數，向上取整以避免門檻本身出現小數次數
6 thresholds["高需求門檻_向上取整"] = np.ceil(
7     thresholds["訓練期80分位數門檻"]
8 ).astype(int)
9 data = data.merge(thresholds, on="站名比對鍵", how="left", validate="many_to_one")
10 # 借車次數達到（或超過）該站自己的門檻，才算高需求：每站門檻不同，是「站點相對」的定義
11 data["是否高需求"] = (data["借車次數"] >= data["高需求門檻_向上取整"]).astype(int)
```

模型看什麼？

X是9個特徵

1

前4個是類別型：站點、星期幾、是否週末、天氣現象代碼

後5個是數值型：小時_sin、小時_cos、溫度、降雨量、風速）。

Y是預測目標

2

也就是這個站點這個小時是不是高需求，答案只有1或0兩種。

任務類型 - 二元分類，模型

3

判斷的就是「是」或「不是高需求」。

```
step04_classification_model_baseline.py

1  # 模型實際使用的 9 個特徵：站點識別、時間類特徵、天氣類特徵，皆為訓練與測試期都能取得的欄位
2  feature_columns = [
3      "站名比對鍵", "星期幾", "是否週末", "weather_code", "小時_sin", "小時_cos",
4      "temperature_2m", "rain", "wind_speed_10m",
5  ]
6
7  # 三個比較對象：多數類別基準（永遠猜非高需求）、Logistic Regression、Random Forest
8  models = {
9      "多數類別基準": DummyClassifier(strategy="most_frequent"),
10     "Logistic Regression": LogisticRegression(
11         max_iter=1000, class_weight="balanced", random_state=42
12     ),
13     "Random Forest": RandomForestClassifier(
14         n_estimators=250, min_samples_leaf=3,
15         class_weight="balanced_subsample", n_jobs=-1, random_state=42,
16     ),
17 }
```

用了哪些模型？怎麼測試？

時間軸 訓練期 2025/12–2026/01 (31,248筆) 測試期 2026/02 (14,112筆)
按時間切分、非隨機打散，才能反映模型面對「沒看過的月份」的真實預測能力

1

訓練期是2025年12月到2026年1月，共31,248筆；測試期是2026年2月，共14,112筆。

2

我們採用按時間先後切分的方式，用前兩個月訓練、最後一個月測試，這樣才能反映模型面對「沒看過的月份」時真正的預測能力。

3

類別權重平衡：因為高需求的資料比較少，我們兩個分類模型都做了類別權重平衡處理。

4

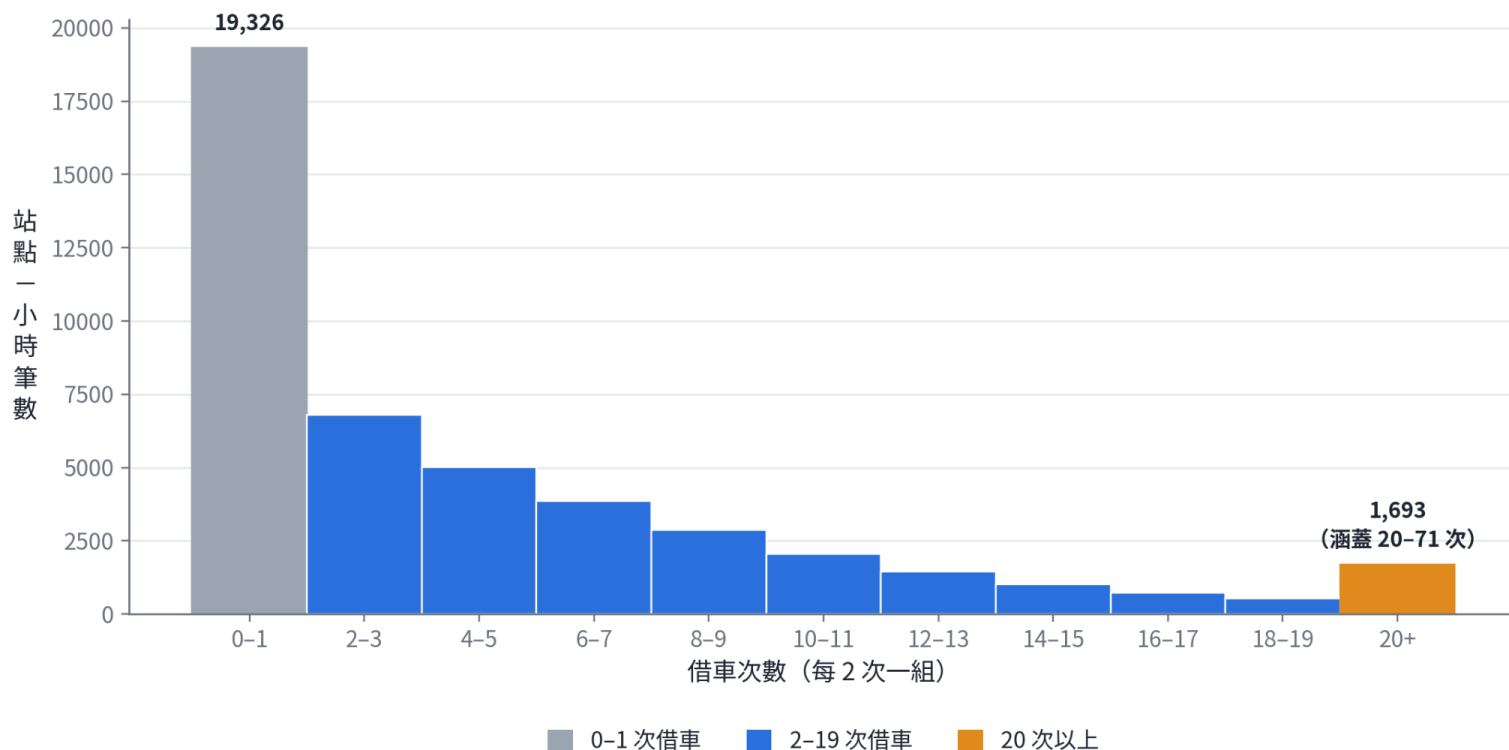
隨機森林用的是專屬的「子抽樣平衡」版本，因為隨機森林是由多棵樹在不同抽樣子集上訓練，平衡方式和邏輯回歸略有不同，但目的是一致的。

```
step03_high_demand_labeling.py

1  # 取最後一個月作為測試期，前面所有月份作為訓練期
2  test_month = months[-1]
3  # 依時間切分，不隨機打散：最後一個月完全保留為測試期，避免未來月份混入訓練期造成資料洩漏
4  data["資料切分"] = np.where(
5      data["時間"].dt.to_period("M") == test_month,
6      "測試期",
7      "訓練期",
8  )
```

借車次數的分布長什麼樣？

每站每小時借車次數分佈：多數時段借車量低，高次數為少數



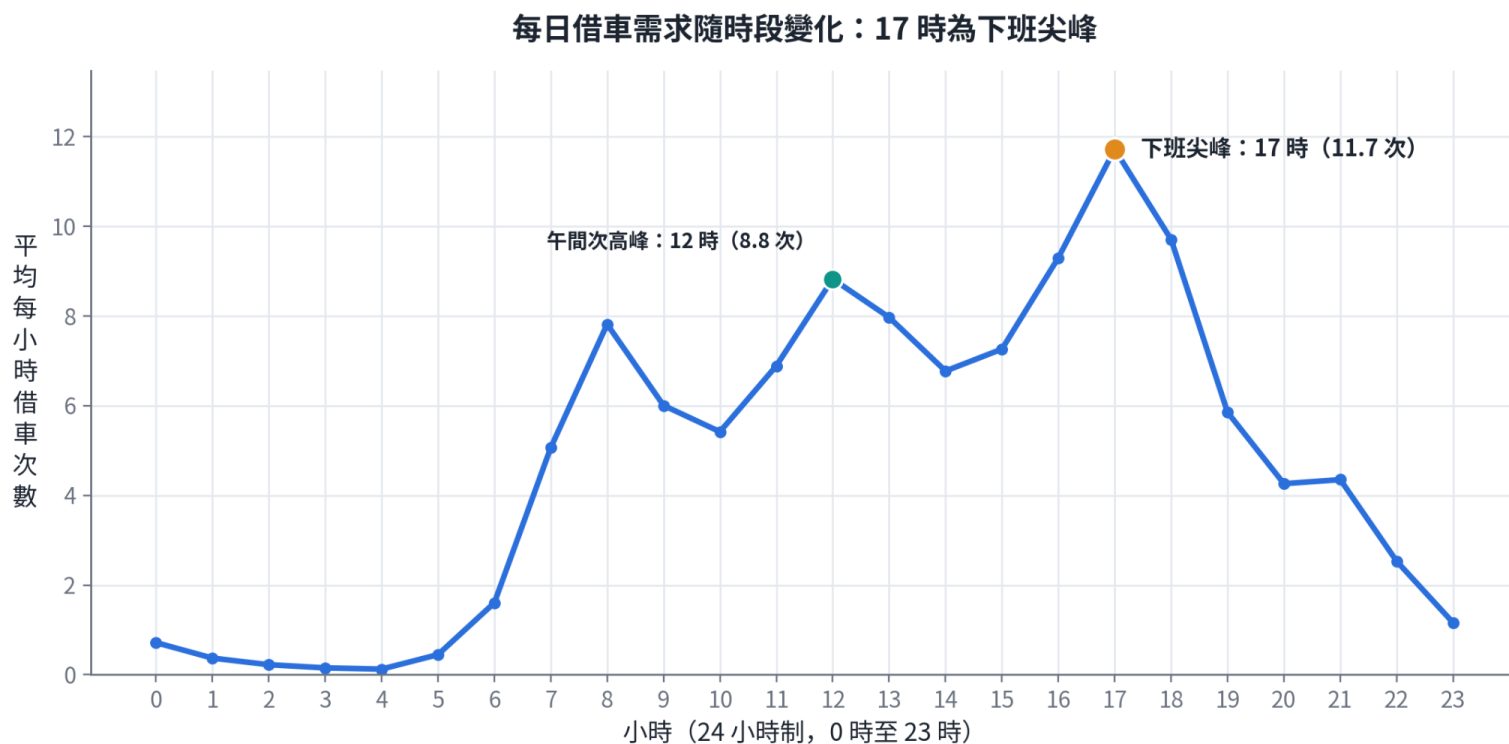
1

左圖顯示，高需求本來就是少數情況，訓練期占 22.48%，測試期占 14.23%。

2

這也是為什麼我們選擇用精確率、召回率這些指標，來更完整地判斷模型好壞，而不是只看類別不平衡下容易失真的整體答對率。

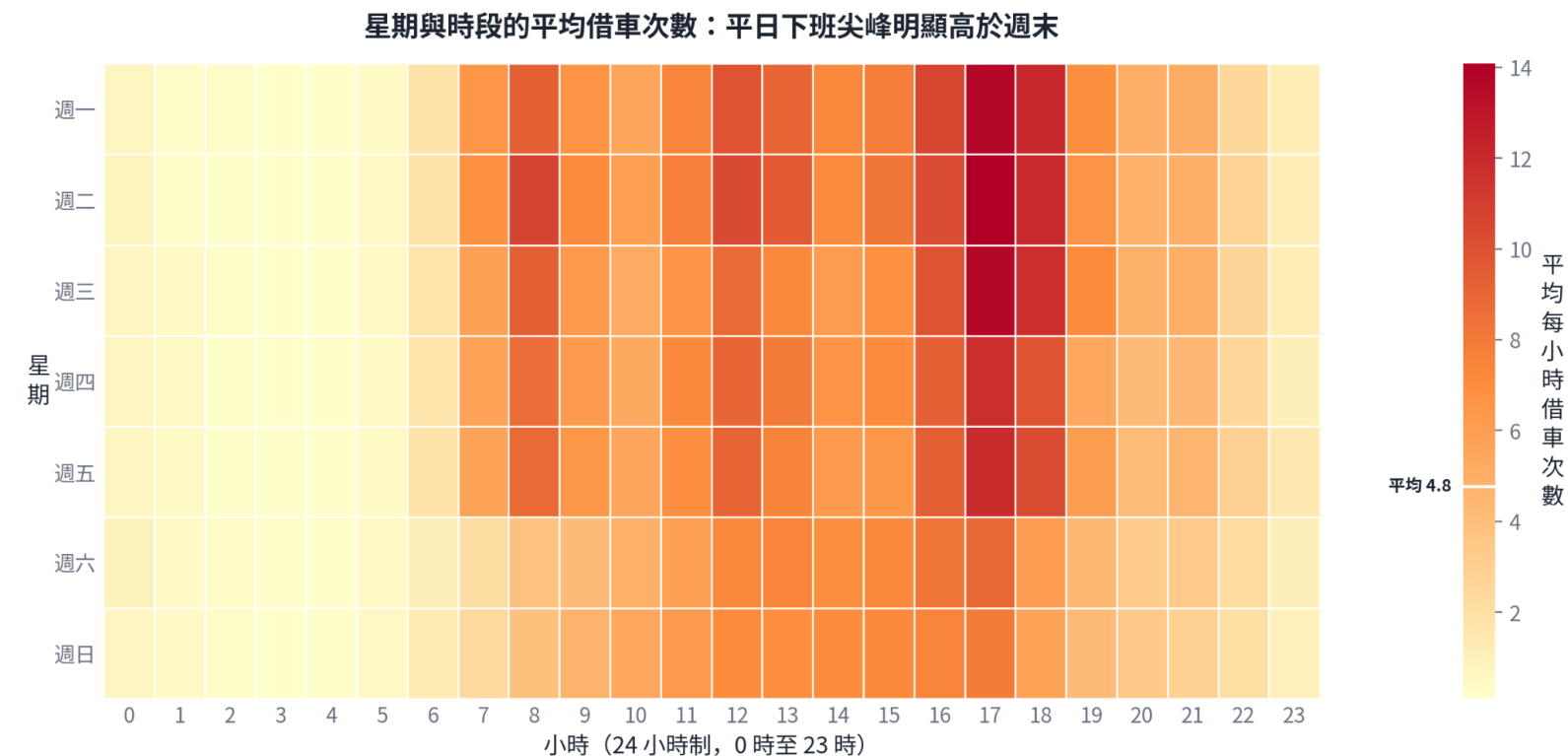
借車次數的分布長什麼樣？(續)



3

此圖顯示整體平均借車次數在17時，也就是下午5點，達到最高峰。

星期和時段合起來看，型態長怎樣？



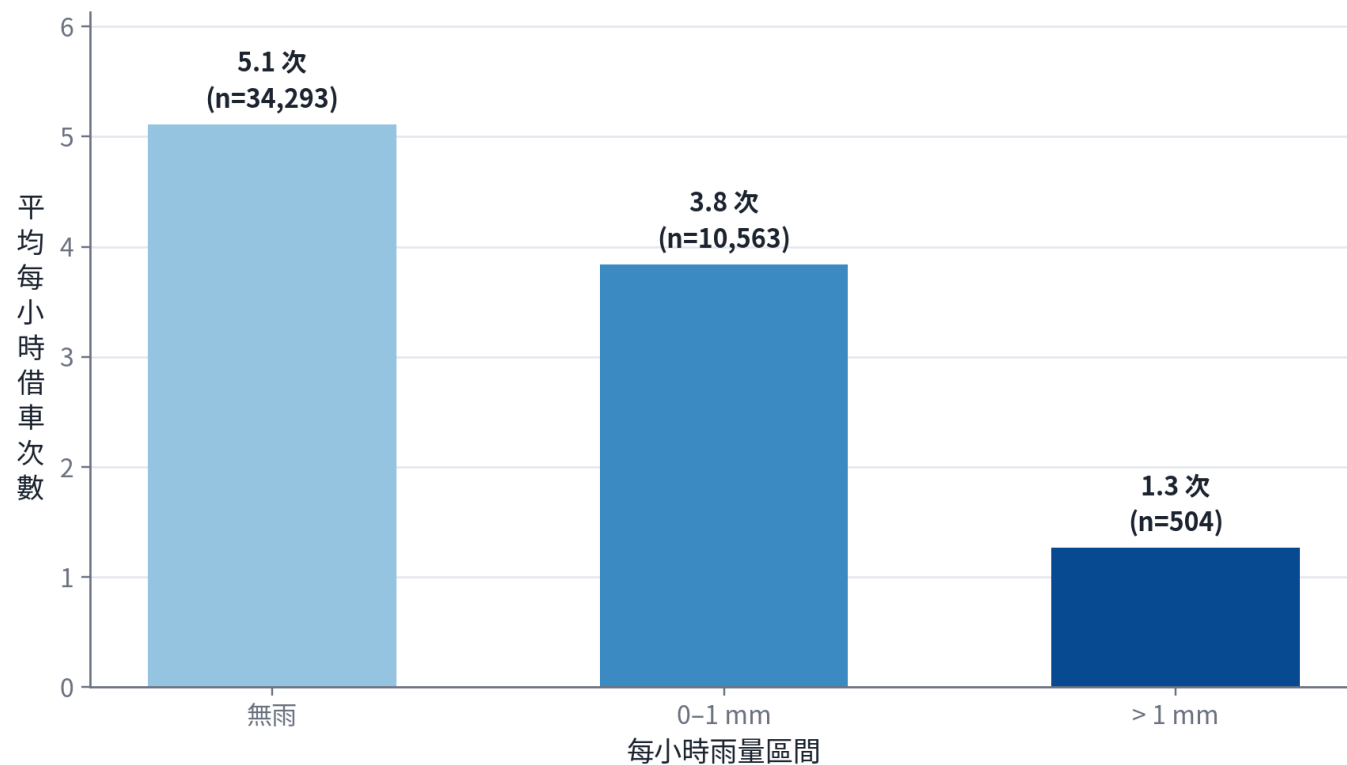
1 這張熱圖同時呈現星期和時段兩個維度：顏色越深，代表該星期、該時段的平均借車次數越高。

2 可以看到週間傍晚（約16時到18時）的顏色明顯較深，需求比較集中；週末同樣時段的顏色明顯較淺，型態跟週間不一樣。

3 這張圖呈現的是使用型態，不代表課表或活動就是唯一原因，站點、天氣等因素也可能同時影響。

天氣跟借車量有關係嗎？

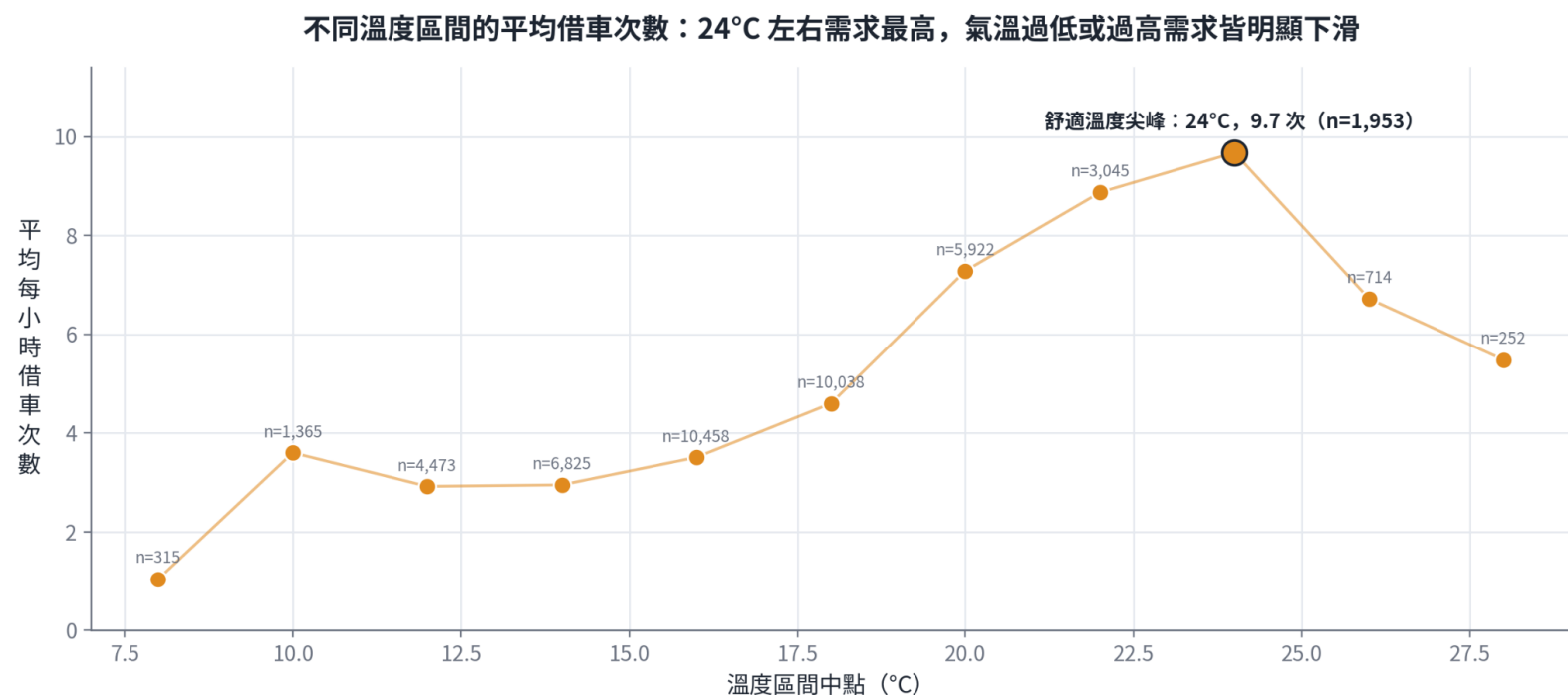
不同雨量區間的平均借車次數：雨量 > 1mm 時需求驟減75%，僅剩無雨時的四分之一



1

左圖是雨量，可以看到雨量較大時平均借車次數確實比較低。

天氣跟借車量有關係嗎？(續)



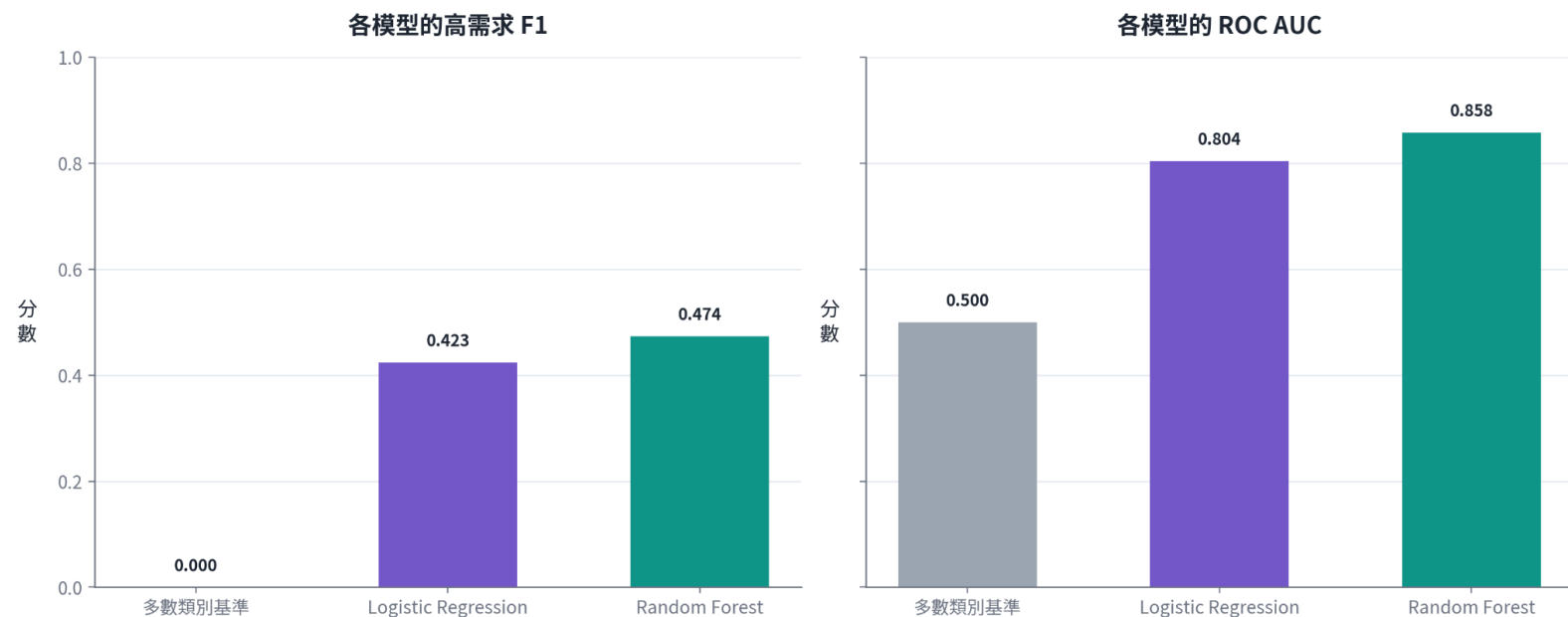
2

此圖是溫度，以每2.5°C為一個區間呈現平均借車次數：從7°C到16°C左右，平均借車次數都偏低，大約1.4到4.1次；溫度往上到20°C到24°C之間，平均借車次數明顯升高，最高落在22°C到24°C區間，平均達9.3次；超過26°C之後平均借車次數又往下降，但這幾個高溫區間的樣本數比較少，這段趨勢僅供參考。

這兩張圖用的都是區間平均。

3

模型表現得怎樣？



附注。分數皆為 0 至 1，數值越高代表模型表現越好；F1 為測試期「高需求」類別的 F1 分數，ROC AUC 為測試期整體區辨能力。
灰色長條為多數類別基準；紫色＝Logistic Regression、青色＝Random Forest（配色與圖 10 一致，便於跨圖對照同一模型）。
多數類別基準的高需求 F1 恰為 0.000：因為此基準永遠不會預測「高需求」，該類別的精確率與召回率皆為 0，數學上必然如此，並非圖表或計算錯誤。

1

多數類別基準的準確率高達**0.8577**，但它的做法是把每筆資料都預測成「非高需求」，這說明了為什麼這一題要優先看精確率、召回率這些更能反映實際判斷能力的指標。

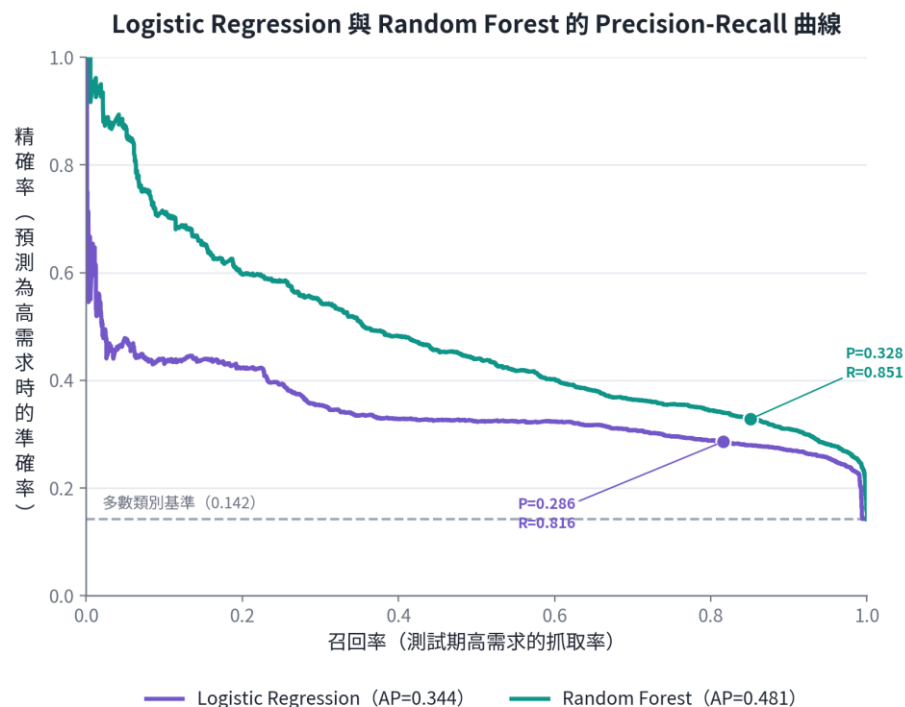
2

隨機森林的F1分數與ROC AUC皆為三者最高，我們選它作為主要模型。

模型表現得怎樣？

模型	準確率	精確率	召回率	F1分數	ROC AUC
多數類別基準	0.8577	0.0000	0.0000	0.0000	0.5000
邏輯回歸(Logistic Regression)	0.6842	0.2860	0.8152	0.4235	0.8039
隨機森林(Random Forest)	0.7311	0.3283	0.8506	0.4737	0.8575

精確率0.3283，實際上是什麼意思？



附注：圓點為目前實際使用的判斷門檻（機率 0.5）對應位置；
AP (Average Precision) 為曲線下面積，數值越高代表模型在各種門檻下的整體表現越好；
虛線為資料中高需求類別的實際比例（多數類別基準）。

這張圖的橫軸是召回率（實際抓到多少真高需求）、縱軸是精確率（提醒的裡面有多少是真的），圖上有隨機森林（青色）和邏輯回歸（紫色）兩條線，各自標出我們目前實際使用的0.5門檻對應的位置。

精確率0.3283，實際上是什麼意思？(續)

2

隨機森林在0.5門檻的位置是精確率0.328、召回率0.851：換算成白話，模型每提醒3次「可能高需求」，大約1次真的達到門檻；同時，真正的高需求時段裡，有85%都被它抓到，漏掉的只有15%。

3

邏輯回歸在0.5門檻的位置是精確率0.286、召回率0.816，兩個數字都比隨機森林低，這也是我們選用隨機森林作為主要模型的原因之一。

4

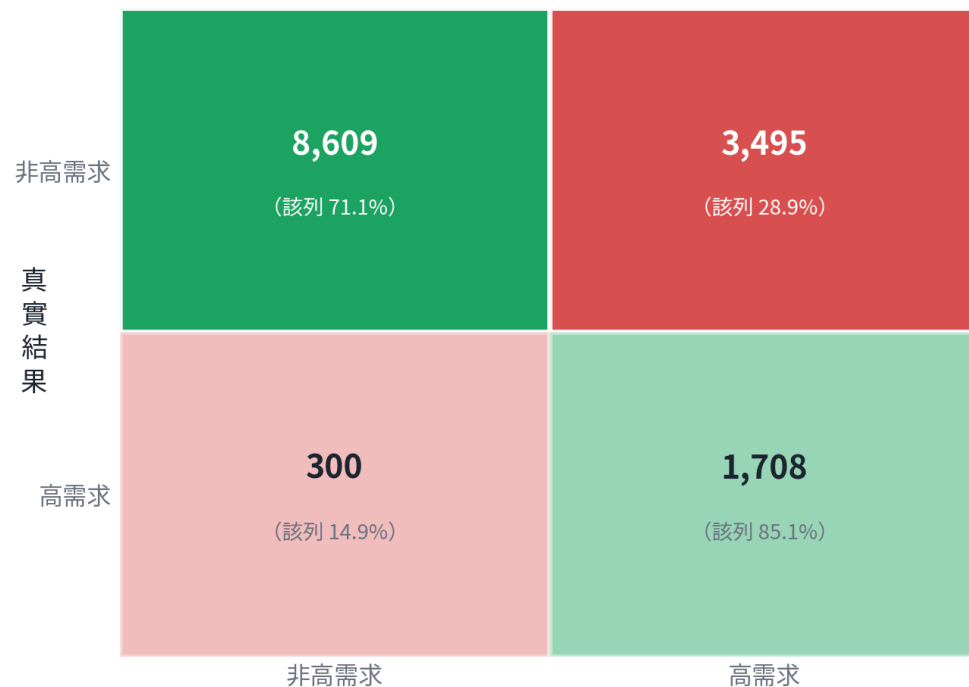
圖例裡的AP（平均精確率）是把「所有可能門檻」都算進去的整體表現，跟前面講的0.328、0.286是「只看0.5這一個門檻」不一樣：隨機森林的AP是0.481，邏輯回歸是0.344，隨機森林在各種門檻下都優於邏輯回歸。圖上最下面那條虛線是0.142，代表如果什麼都不篩選、每個時段都亂猜，精確率大概只有這個水準，兩個模型都遠遠超過這條基準線，說明模型是真的有在學到東西，不是瞎猜。

5

精確率不高不代表模型沒用：這個模型完整達成了分類模型建立與評估的目標，適合作為歷史需求傾向的參考依據，實際的判對、判錯筆數，下一頁用混淆矩陣攤開來看。

模型實際判對、判錯了多少？

Random Forest 在保留測試期的判別結果



附注。單位為站點一小時筆數，測試期共 14,112 筆。

綠色：答對（含成功抓到與正確排除）；紅色：答錯（漏掉或誤報）；顏色深淺對應筆數多寡。

- 1 8,609筆是真實非高需求，模型也預測為非高需求。
- 2 1,708筆是真實高需求，模型也成功預測為高需求。
- 3 300筆是真實高需求，但模型漏掉了。
- 4 3,495筆是模型提醒高需求，但實際上沒有達到門檻，這是誤報。
- 5 整體來說，模型比較擅長抓到高需求時段，但誤報仍然偏多，我們把它定位為「歷史需求傾向參考」，提供借車人提早規劃的參考依據。

答案：高需求集中在這些時段

平日的通勤與午餐尖峰時段最容易高需求，尤其是週一到週三的17時前後；週末和深夜幾乎不會出現高需求。

1

先看時段：17時（下午5點）機率最高，達到61.2%；其次是18時48.7%、12時47.4%、16時45.2%、13時41.0%、8時38.6%，也就是上下班上下課的通勤尖峰，再加上中午吃飯時段。

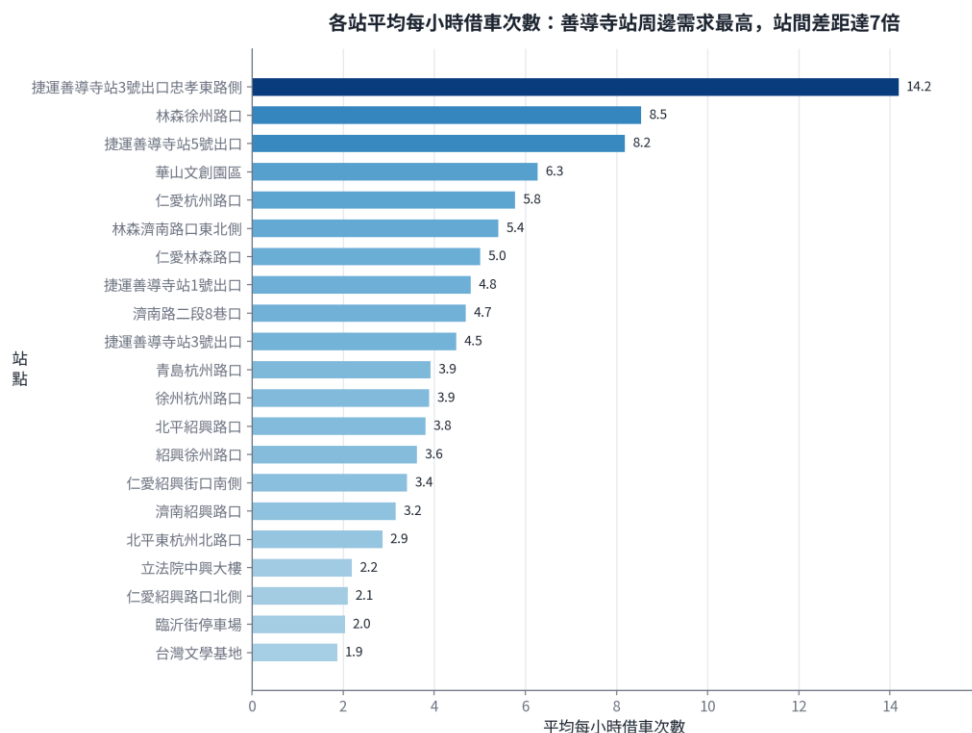
2

再看星期：平日明顯高於週末，其中週二最高（26.2%），週一次之（24.7%）；週六只有12.6%、週日只有11.5%，週末的高需求機率大約只有平日的一半。

3

把兩者疊加來看，平日尖峰時段（如上下班、中午）高需求機率高達48.0%，週末同樣是尖峰時段的鐘點，機率則是22.1%——時段和星期要合併判斷，才能掌握最準確的高需求組合。

答案：高需求集中在這些時段



4

這張長條圖是各站的平均每小時借車次數排名：最高的是捷運善導寺站3號出口（忠孝東路側），平均每小時14.2次；最低的是臺灣文學基地，平均每小時只有1.9次，兩者差距約7.6倍，可以看出站點之間的需求量落差其實很大。

5

換成「被標記為高需求的比例」來看，站點之間的差距就縮小很多，但「高需求」門檻本來就是每一站用自己的80分位數去算的，所以借車量本來就少的站，也一樣有屬於自己的相對尖峰時刻。

6

天氣也有影響：有降雨時的高需求機率是13.1%，沒降雨時是22.1%，方向與第18頁的觀察一致，這是我們資料呈現的關聯趨勢。

我們做到了什麼？

我們用歷史借車紀錄結合時段、星期與天氣資料，成功建立並評估了北商周邊YouBike站點的高需求時段預測模型，最佳模型為隨機森林，F1分數0.4737，ROC AUC 0.8575，能辨識出部分相對高需求時段，答案就是上一頁列出的平日通勤與午餐尖峰。

1

整個流程從資料蒐集、清理、特徵建立，一路做到模型評估，而且用三個月沒看過的資料驗證了模型的泛化能力。

2

模型的部分，我們建立了3個並排比較，包括多數類別基準、邏輯回歸與隨機森林，完整呈現不同方法的表現差異。

3

測試方式採用時間切分，確保結果反映的是「面對未來月份」的真實預測能力。

4

精確率偏低、誤報偏多這些限制，我們也都完整如實揭露，讓這份報告的每個數字都經得起檢驗。

限制與未來方向

1

資料僅三個月、集中冬季 → 未來延長月份，涵蓋不同季節

2

80分位數未做敏感度分析 → 未來比較不同分位數門檻

3

站點範圍以目前即時站點位置回推三個月前的歷史範圍 → 未來用歷史期間內的站點紀錄重新確認

4

天氣資料為最近氣象網格點，與北商中心約差2公里 → 屬資料先天限制

5

未使用即時可借車數 → 本題僅為需求傾向參考，非即時保證

參考資料

研究文獻

1

(第一篇 · 紐約研究) Faghih-Imani, A., & Eluru, N. (2016). Incorporating the impact of spatio-temporal interactions on bicycle sharing system demand: A case study of New York CitiBike system. *Journal of Transport Geography*, 54, 218–227.

2

(第二篇 · 多倫多研究) El-Assi, W., Salah Mahmoud, M., & Habib, K. N. (2017). Effects of built environment and weather on bike sharing demand: A station level analysis of commercial bike sharing in Toronto. *Transportation*, 44(3), 589–613.

3

(第三篇 · 舊金山灣區研究) Ashqar, H. I., Elhenawy, M., & Rakha, H. A. (2019). Modeling bike counts in a bike-sharing system considering the effect of weather conditions. *Case Studies on Transport Policy*, 7(2), 261–268.

資料來源

- 臺北市政府資料大平臺——YouBike2.0 即時資訊與歷史租借紀錄
- Open-Meteo——Historical Weather API

謝謝老師的指導 與同學們的聆聽

第02組 | 專題報告完畢

組長：賴子渝 組員：王秀方 指導老師：江尚瑀 老師

